

2D Array

Date: / / Page no: _____

□ Introduction To 2-D Array :-

➤ **Multidimensional Array :-** Multidimensional array are used to store the data in rows and columns, where each row can represent another individual array. It is also known as array of arrays.

• Two-Dimensional Array (2D Array) :-

Two-Dimensional array is the simplest form of a multidimensional array. A 2-D Array can be seen as an array storing multiple 1-D array.

• Syntax :-

// Declaring and Initializing

data_type [][] arr_name = new data_type[x][y]

// Assigning Value

arr_name[row-index][column-index] = value;

□ Representation of 2-D Array :-

A 2D array can be seen as a table with 'x' rows and 'y' columns where the row

number ranges from 0 to $(x-1)$ and column number ranges from 0 to $(y-1)$. A 2D array of 'x' with 3 rows & 3 columns are shown below:-

	Column 0	Column 1	Column 2
Row 0	$x[0][0]$	$x[0][1]$	$x[0][2]$
Row 1	$x[1][0]$	$x[1][1]$	$x[1][2]$
Row 2	$x[2][0]$	$x[2][1]$	$x[2][2]$

rows = 3

 $n \times m$

column = 3

↓ ↓

Rows Columns

⇒ Total number of cell in 2D = rows $\times$ columns

□ Real life example of 2-D Array:

- Spreadsheets (like Excel).
- Chessboard.
- Traffic management System
- Image processing (Pixel's colour setting)
- Setting in rgb in Image

□ Creation of 2-D Array

→ import java.util.*;

```
public class Creation {
```

// Searching Largest element in 2D Array

```
public static void Largest (int matrix[][]) {
```

```
    int p = Integer.MIN-Value;
```

```
    for (int i=0; i < matrix.length; i++) {
```

```
        for (int j=0; j < matrix[0].length; j++) {
```

```
            if (matrix[i][j] > p) {
```

```
                p = matrix[i][j];
```

```
            }
```

```
        }
```

```
    }
```

```
    } sopLn("Largest Element is: " + p);
```

// Searching Smallest element in 2D Array

```
public static void Smallest (int matrix[][]) {
```

```
    int t = Integer.MAX - Value;
```

```

for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[0].length; j++) {
        if (matrix[i][j] < t) {
            t = matrix[i][j];
        }
    }
}
Sopln ("Smallest element is : " + t);
}

```

// Searching an element in 2D Array

```

public static boolean Search (int matrix[][], int Key)
{
    for (int i = 0; i < matrix.length; i++) {
        for (int j = 0; j < matrix[0].length; j++) {
            if (matrix[i][j] == Key) {
                Sopln ("Found at: (" + i + ", " + j + ")");
                return true;
            }
        }
    }
    S.opln ("Not Found");
    return false;
}

```

```
public static void main (String [] args) {  
    Scanner sc = new Scanner (System.in);  
  
    int matrix [][] = new int [3][3];  
  
    int n = matrix.length;  
    int m = matrix[0].length;  
  
    Sopln ("Enter element:");  
  
    // Taking input  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            matrix[i][j] = sc.nextInt();  
        }  
    }  
  
    // Display  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            Sopln (matrix[i][j] + " ");  
        }  
        Sopln ("");  
    }  
  
    Search (matrix, 64);  
    largest (matrix);  
    smallest (matrix);  
}
```

• Output :- Enter Element :

91 32 54 23 64 22 99 76 34

91	32	54
23	64	22
99	76	34

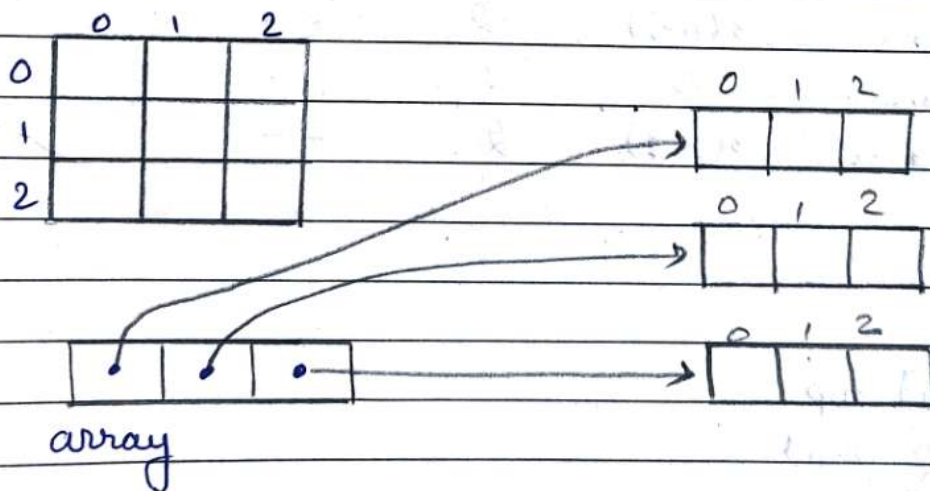
Found at : (1,1);

Largest element is : 99

Smallest element is : 91

□ 2D Array in Memory :-

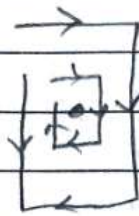
In Java the elements of array are not stored in row major and column major.



the arrays of rows are stored anywhere in the memory.

□ Spiral Matrix :-

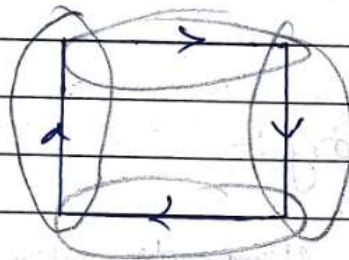
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16



Spiral.

Output:- 1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10

Approach :-



2 col
2 row } in one loop

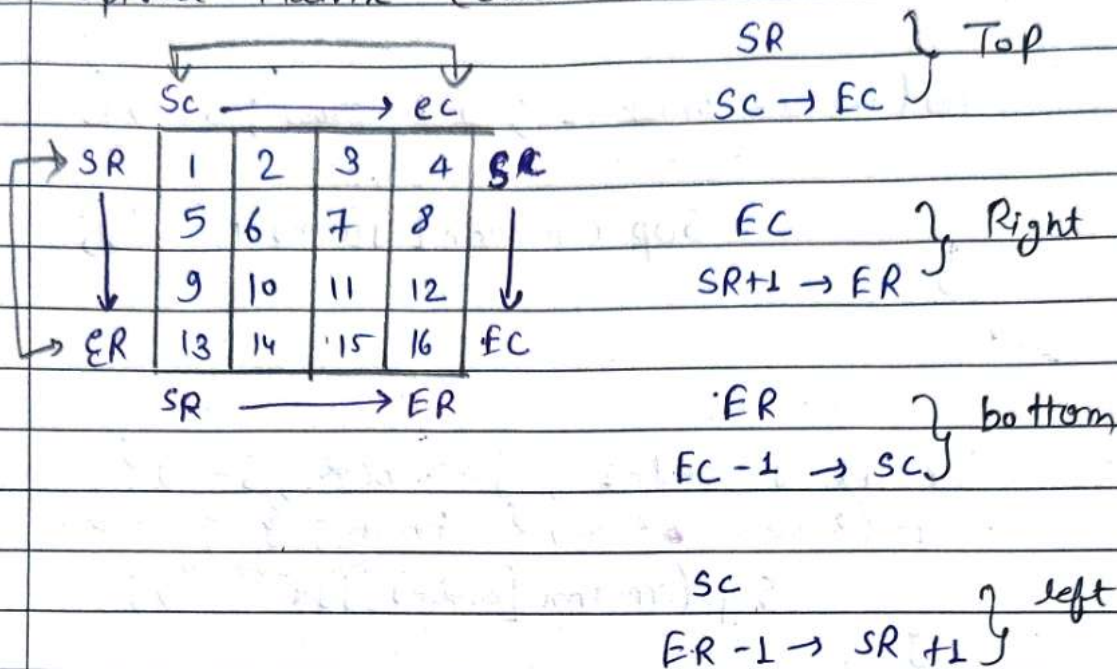
	1 st	2 nd	
Start Row	0	1	++
end Row	3(n-1)	2	--
Start col	0	1	++
end col	3(n-1)	2	--

while ()
{

- ① top
- ② right
- ③ bottom
- ④ left

}

Spiral Matrix Code:



→ Code

```

public class SpiralMatrix {
    + public static void SM (int matrix [] []) {
        int srow = 0;
        int scol = 0;
        int erow = matrix.length - 1;
        int ecol = matrix[0].length - 1;

        while (srow <= erow && scol <= ecol) {
            //top
            for (int j = scol; j < ecol; j++) {
                S.op (matrix [srow] [j]) + " ";
            }
        }
    }
  
```

//right

```
for(int i = srow + 1; i <= erow; i++) {
    Sop (matrix[i][ecol] + " ");
}
```

//bottom

```
for(int j = ecol - 1; j >= scol; j--) {
    if(srow == erow) { break; } } Return;
    Sop (matrix[erow][j] + " ");
}
```

//left

```
for(int i = erow - 1; i >= srow + 1; i--) {
    if(scol == ecol) { break; } } Return;
    Sop (matrix[i][scol] + " ");
}
```

	Output :-	1	2	3	4	5	8	7	12	16
srow ++;					15	14	13	9	5	6
erow --;				7	11	10				
scol ++;										
ecol --;										
}										
}										

```
public static void main (String [] args) {
```

```
    int matrix[][] = { {1, 2, 3, 4}, {5, 6, 7, 8},
                       {9, 10, 11, 12}, {13, 14, 15, 16} }
```

```
    SM (matrix);
```

```
}
```

[Amazon, Microsoft, Samsung]

■ Diagonal Sum: ($n = \text{even}$)

 $n = m$

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

 $i+j = n-1$

34

(Secondary diagonal)

 $i = j$

34 (Primary diagonal)

$$PD = 1 + 6 + 11 + 16 = 34$$

$$SD = 4 + 7 + 10 + 13 = 34$$

$$\text{Diagonal sum} = 68$$

(n = even) :-

(n = odd) ↘

0	1	2
3	4	5
6	7	8

12

12

$$DS = 20 \text{ \{as middle element not include\}}$$

➤ Code :-

```
public class diagonal {
```

```
    public static int dig (int matrix[][]) {
```

```
        int sum = 0;
```

```
        // Code of  $T_c = O(n^2)$ 
```

```
        for (int i = 0; i < matrix.length; i++) {
```

```
            for (int j = 0; j < matrix[0].length; j++) {
```

```

    if (i == j) {
        Sum += matrix[i][i];
    }
    if (i + j == matrix.length - 1) {
        Sum += matrix[i][i];
    }
}
}

```

// more efficient code $TC = O(n)$

```

for (int i = 0; i < matrix.length; i++) {

```

```

    // Primary Diagonal
    Sum += matrix[i][i];

```

```

    // Secondary Diagonal
    if (i != matrix.length - i - 1) {

```

```

        Sum += matrix[i][matrix.length - i - 1];
    }

```

```

    }
    return Sum;
}

```

```

public static void main (String [] args) {

```

```
int mx[][] = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12},
                {13,14,15,16} };
```

```
Sopl<> ("Sum is : " + dig(mx));
```

→ Output :- Sum is : 68

□ Search in Sorted Matrix ($T_c = O(n+m)$)

↳ Search for a Key in row wise & col wise Sorted matrix

APPROACH :-

(BS)

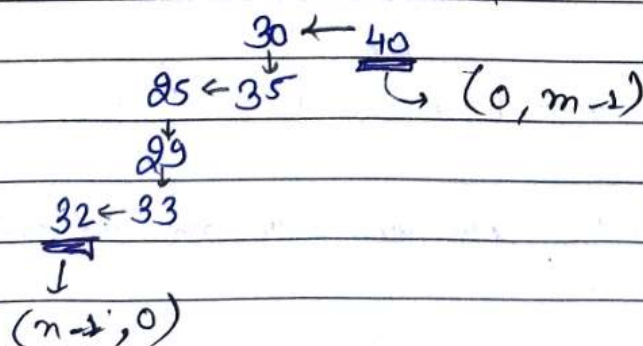
$\times \log(n)$	10	20	30	40
$\times \log(n)$	15	25	35	45
$\times \log(n)$	27	29	37	48
$\times \log(n)$	32	33	39	50

① Brute force (Linear Search)
↳ $O(n^2)$

② Row wise (Binary search)
↳ $n \log n$

③ Stair case search

let Key = 33



TC = $\left. \begin{array}{l} \text{if}(n \gg m) \text{ then } O(n) \\ \text{if}(n \ll m) \text{ then } O(m) \end{array} \right\} O(n+m)$

Date: / / Page no: _____

⇒ In case of Right ~~bottom~~^{Top} cell.

40 ⇒ (0, m-1)

if (Key < cell value)

Left

if (Key > cell value)

Bottom

⇒ In case of Left bottom cell

32 ⇒ (n-1, 0)

if (Key < cell value)

Top

if (Key > cell value)

Right

Raise condition

$\left\{ \begin{array}{l} i = 0 \text{ to } n-1 \\ j = m-1 \text{ to } 0 \end{array} \right\}$

↓

Then Key not found

□ Code For both cases:-

```
public class SearchInSorted2D {
```

```
    public static boolean staircase (int matrix[][], int Key)
```

// In case of Right Top:-

```
/* int row = 0, col = matrix.length - 1;
```

```
while (row < matrix.length && col >= 0) {
```

```
    if (matrix[row][col] == Key) {
```

```
        SoplN("Key found at: (" + row + ", " + col + ")");
        return true;
    }
```

```
    else if (Key < matrix[row][col]) {
```

```
        col --;
```

```
    }
```

```
    else {
```

```
        row ++;
```

```
    }
```

```
}
```

```
SoplN("Key NOT FOUND!");
```

```
return false;
```

```
*/
```

// In case of left Bottom

```
int row = matrix[0].length - 1;
```

```
int col = 0;
```

```
while (col < matrix.length - 1 && row >= 0) {  
    if (matrix[row][col] == key) {  
        Sopln("Key found at: (" + row + ", " + col + ")");  
        return true;  
    }  
    else if (key < matrix[row][col]) {  
        row --;  
    }  
    else {  
        col ++;  
    }  
}  
Sopln("Key not FOUND!");  
return false;  
}
```

```
public static void main (String [] a) {  
    int matrix [][] = { {10, 20, 30, 40}, {15, 25, 35, 45},  
                        {27, 29, 37, 48}, {32, 33, 39, 50} };  
    TestCase (matrix, 37);  
}
```

→ Output :- Key Found at: (2, 2);

Questions:-

- 1) Print the number of 7's that are in 2d array.

Ex:- input:- `int[][] arr = {{4,7,8},{8,8,7}};`

o/p:- 2

Ans:-

```
public class Q1 {

    public static void main (String [] args) {

        int arr[][] = {{4,7,8},{8,8,7}};

        int CountOf7 = 0;

        for (int i=0; arr.length; i++) {
            for (int j=0; j < arr[i].length; j++) {

                if (arr[i][j] == 7) {

                    CountOf7++;

                }

            }

        }

        System.out.println ("Count of 7 is: " + CountOf7);

    }

}
```

→ Output:- Count of 7 is: 2.

Q.2) Print Out the Sum of the numbers in the second row of the "nums" array.

Ex:- Input = int nums[][] = { {1, 4, 9}, {11, 4, 3}, {2, 2, 3} };
O/P = 18

```
public class Q2 {
```

```
    public static void main (String [] args) {
```

```
        int [][] nums = { {1, 4, 9}, {11, 4, 3}, {2, 2, 3} };
```

```
        int sum = 0;
```

```
        for (int j = 0; j < nums[0].length; j++) {
```

```
            sum += nums[1][j];
```

```
        }
```

```
        System.out.println ("sum is : " + sum);
```

```
    }
```

```
}
```

→ Output :- Sum is: 18

3) Write a program to find Transpose of a Matrix.
→ Transpose is Swapping the rows to Column.
For 2x3.

Matrix :-

a ₁₁	a ₁₂	a ₁₃
a ₂₁	a ₂₂	a ₂₃

Transpose : $\begin{matrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{matrix}$

Ans:-

```
public class Q3 {
```

```
    public static void printMatrix (int arr[][]) {
```

```
        System.out.println("The matrix is: ");
```

```
        for (int i=0; i < arr.length; i++) {
```

```
            for (int j=0; j < arr[i].length; j++) {
```

```
                System.out.print(arr[i][j] + " ");
```

```
            }
```

```
            System.out.println();
```

```
        }
```

```
    }
```

```
    public static void main (String [] args) {
```

```
        int [][] matrix = {{1,4,9},{11,4,33}};
```

```
        int row = 2, column = 3;
```

```
        printMatrix (matrix);
```

```
        // New array
```

```
        int [][] Transpose = new int [column][row];
```

```
        for (int j=0; j < matrix[0].length; j++) {
```

```
} Transpose[j][i] = matrix[i][j];
```

```
} print Matrix(Transpose);  
}
```

→ O/p :- The matrix is :-

1 4 9

11 4 3

The matrix after Transpose :-

1 11

4 4

9 3